Probabilistic Machine Learning

Machine Learning course notes Sources: listed under References

What this file is really about:

$$\text{a posterior, not a point} \rightarrow \text{sample what you cannot integrate} \rightarrow \text{approximate} \rightarrow \text{use the uncerta}$$

How to use this file. Part A builds these ideas on one-dimensional examples. Part B shows the core code. Part C is practice with answers.

Step-up path

You need. Bayes' rule, normals and linear regression.

1. Keep a posterior instead of a point.
2. Compute integrals by sampling.
3. Build chains with MCMC.
4. Approximate with a simple family.
5. Use the uncertainty to act.

You can stop when you can say what you are unsure about, and use it.

Contents

Part A Maths From the Ground Up

A1 Why a posterior

MLE gives one best θ. Bayes' rule gives a distribution over θ:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}.$$

Predictions then average over what we are unsure about:

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, \theta)\, p(\theta \mid \mathcal{D})\, d\theta.$$

MAP keeps only the peak of the posterior. A Gaussian prior gives ridge regression, and a Laplace prior gives lasso.

A2 Bayesian linear regression

With prior $\theta \sim \mathcal{N}(0, \alpha^{-1} I)$ and noise precision β, the posterior is Gaussian:

$$S^{-1} = \alpha I + \beta X^\top X, \qquad m = \beta S X^\top y.$$

The predictive variance is $\beta^{-1} + x^\top S x$: noise plus uncertainty about the weights. It grows away from the data.

A3 Monte Carlo

When an integral is hard, sample:

$$\mathbb{E}[f(x)] \approx \frac{1}{N} \sum_i f(x_i).$$

The error shrinks like $1/\sqrt{N}$, whatever the dimension. Estimating π from points in a square is the classic example.

A4 Sampling methods

- **Inverse CDF.** If u is uniform, then $F^{-1}(u)$ has distribution F.
- **Rejection sampling.** Propose from q and accept with probability $p(x)/(Mq(x))$.
- **Importance sampling.** Reweight samples from q by $w = p/q$.

A5 Markov chain Monte Carlo

Metropolis-Hastings builds a chain whose long-run distribution is the target. Propose a step, and accept with

$$A = \min\left(1, \frac{p(x')}{p(x)}\right)$$

for a symmetric proposal. Only ratios appear, so the unknown normalising constant cancels. Small steps explore slowly, and large steps are mostly rejected. **Gibbs sampling** updates one variable at a time from its conditional.

A6 Variational inference

Instead of sampling, pick a simple family $q_\phi(\theta)$ and maximise the same evidence lower bound as the VAE. It is fast, but a Gaussian q cannot represent a two-peaked posterior and tends to underestimate spread. The **Laplace approximation** puts a Gaussian at the MAP point, using the curvature there.

A7 Gaussian processes

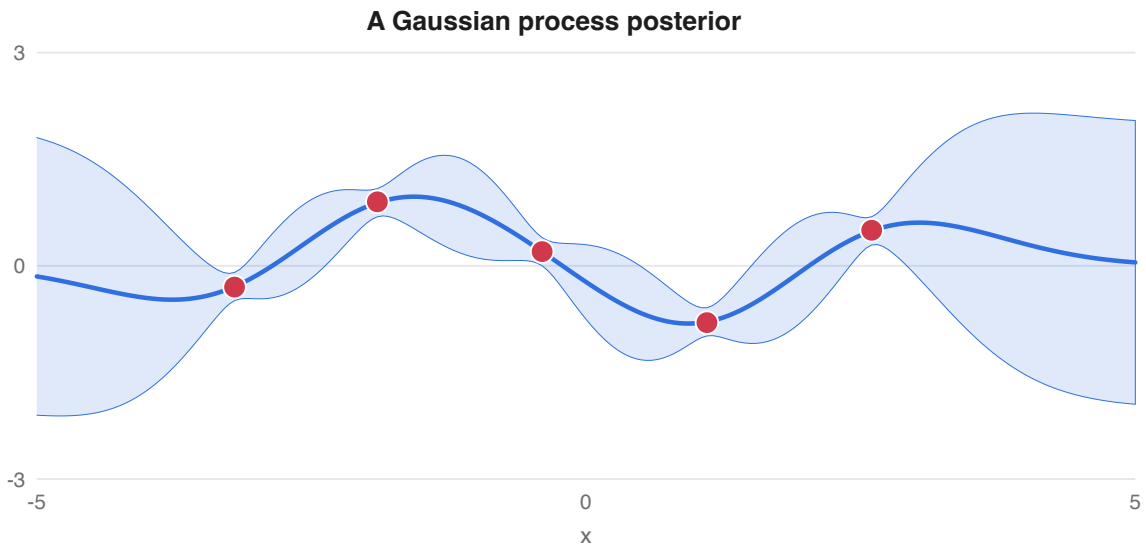
A **Gaussian process** places a prior on functions directly: any finite set of function values is jointly Gaussian, with covariance given by a kernel such as

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right).$$

Conditioning on data gives the posterior in closed form:

$$\mu_* = K_*^\top (K + \sigma_n^2 I)^{-1} y, \qquad \Sigma_* = K_{**} - K_*^\top (K + \sigma_n^2 I)^{-1} K_*.$$

The cost is $O(n^3)$.



The band is narrow at the observations and widens away from them.

A8 Using uncertainty

Aleatoric uncertainty is noise in the data, which more data will not remove. **Epistemic** uncertainty is lack of knowledge, which more data can reduce. **Deep ensembles** and **Monte Carlo dropout** estimate the second from the disagreement of many predictions.

Active learning asks for labels where the model is least certain. **Bayesian optimisation** picks the next point to try using an acquisition function, such as the upper confidence bound $\mu(x) + \kappa\sigma(x)$, which trades exploiting good regions against exploring unknown ones.

Reliability diagrams compare predicted confidence with observed frequency.

A9 The Bayesian linear regression posterior, derived

With prior $w \sim \mathcal{N}(0, \alpha^{-1} I)$ and likelihood $y \sim \mathcal{N}(Xw, \beta^{-1} I)$, the log posterior is, up to constants,

$$-\frac{\beta}{2}\|y - Xw\|^2 - \frac{\alpha}{2}\|w\|^2 = -\tfrac{1}{2} w^\top (\beta X^\top X + \alpha I) w + \beta y^\top X w + \text{const}.$$

A Gaussian with mean m and covariance S has log density $-\tfrac{1}{2} w^\top S^{-1} w + m^\top S^{-1} w + \text{const}$. Matching terms gives

$$S = (\alpha I + \beta X^\top X)^{-1}, \qquad m = \beta S X^\top y.$$

The predictive variance at x_* is $\beta^{-1} + x_*^\top S x_*$: noise, plus uncertainty about the weights.

A10 The Gaussian process prediction, derived

Training outputs and a test output are jointly Gaussian:

$$\begin{bmatrix} y \\ f_* \end{bmatrix} \sim \mathcal{N}\Big(0, \begin{bmatrix} K + \sigma^2 I & k_* \\ k_*^\top & k_{**} \end{bmatrix}\Big).$$

The standard formula for a conditional Gaussian gives

$$\mathbb{E}[f_* \mid y] = k_*^\top (K + \sigma^2 I)^{-1} y, \qquad \mathrm{Var}[f_* \mid y] = k_{**} - k_*^\top (K + \sigma^2 I)^{-1} k_*.$$

Inversion costs $O(n^3)$, so exact models suit up to a few thousand points.

A11 Why Metropolis-Hastings works

A chain with transition T leaves π unchanged if **detailed balance** holds: $\pi(x)T(x \to y) = \pi(y)T(y \to x)$. With a symmetric proposal and acceptance $a = \min(1, \pi(y)/\pi(x))$, take $\pi(y) \le \pi(x)$. Then

$$\pi(x) \cdot \frac{\pi(y)}{\pi(x)} = \pi(y) \cdot 1.$$

Only the ratio appears, so the normalising constant is never needed.

A12 Variational inference, derived

Choose a family q and maximise the ELBO, which is equivalent to minimising $\mathrm{KL}(q \| p(\cdot \mid \mathcal{D}))$. A **mean-field** family factorises, $q(w) = \prod_i q_i(w_i)$, and so ignores correlations. This tends to underestimate the variance.

A13 Gibbs sampling

For a joint distribution, repeatedly sample each variable from its conditional, given all the others. It is Metropolis-Hastings with acceptance always equal to 1. For a bivariate normal with correlation ρ, $x \mid y \sim \mathcal{N}(\rho y, 1-\rho^2)$.

Part B In Code

B1 Bayesian linear regression

```
S = np.linalg.inv(alpha * np.eye(d) + beta * X.T @ X)
m = beta * S @ X.T @ y
```

B2 Metropolis

```
prop = x + step * rng.normal()
if rng.random() < min(1, p(prop) / p(x)):
    x = prop
samples.append(x)
```

B3 Gaussian process regression

```
import numpy as np

def rbf(a, b, ell=1.0):
    return np.exp(-0.5 * (a[:, None] - b[None]) ** 2 / ell ** 2)

def gp_predict(X, y, Xs, noise=0.1, ell=1.0):
    K = rbf(X, X, ell) + noise ** 2 * np.eye(len(X))
    L = np.linalg.cholesky(K)                       # stable, unlike a direct inverse
    alpha = np.linalg.solve(L.T, np.linalg.solve(L, y))
    ks = rbf(X, Xs, ell)
    v = np.linalg.solve(L, ks)
    return ks.T @ alpha, 1 - (v ** 2).sum(0)

X = np.linspace(-3, 3, 12); y = np.sin(X)
mean, var = gp_predict(X, y, np.array([0.1, 10.0]))
print('near the data', mean[0].round(3), var[0].round(4), '| far away', mean[1].round(3), v
assert abs(mean[0] - np.sin(0.1)) < 0.05 and var[1] > 0.99
```

B4 Gibbs sampling

```
import numpy as np

rng = np.random.default_rng(0)
rho, x, y, s = 0.8, 0.0, 0.0, []
for i in range(20000):
    x = rng.normal(rho * y, np.sqrt(1 - rho ** 2))
    y = rng.normal(rho * x, np.sqrt(1 - rho ** 2))
    s.append((x, y))
c = np.corrcoef(np.array(s[1000:]).T)[0, 1]
print('sample correlation', round(c, 3))
assert abs(c - rho) < 0.02
```

B5 Mean-field variational inference by gradient

```
import torch

torch.manual_seed(0)
X = torch.randn(40, 2); y = X @ torch.tensor([2.0, -1.0]) + 0.5 * torch.randn(40)
mu = torch.zeros(2, requires_grad=True); rho = torch.zeros(2, requires_grad=True)
opt = torch.optim.Adam([mu, rho], lr=0.02)

for step in range(3000):
    sd = torch.nn.functional.softplus(rho)
    w = mu + sd * torch.randn(32, 2)
    like = -0.5 * ((y[:, None] - X @ w.T) ** 2).sum(0) / 0.25
    kl = 0.5 * (mu ** 2 + sd ** 2 - 2 * sd.log() - 1).sum()
    loss = -like.mean() + kl
    opt.zero_grad(); loss.backward(); opt.step()

print('posterior mean', mu.detach().numpy().round(2), 'sd', torch.nn.functional.softplus(rh
assert abs(mu[0] - 2) < 0.4 and abs(mu[1] + 1) < 0.4
```

Part C Practice

P1. With a Gaussian prior on the weights, MAP corresponds to which regression?

Ridge regression.

P2. How does the Monte Carlo error change if N is multiplied by 100?

It shrinks by a factor of 10.

P3. $p(x) = 0.2$, $p(x') = 0.1$, symmetric proposal. Find the acceptance probability.

0.5.

P4. Why does MCMC not need the normalising constant?

Only ratios of p appear, so the constant cancels.

P5. $\mu = 2$, $\sigma = 0.5$, $\kappa = 2$. Find the upper confidence bound.

3.

P6. Will more data reduce aleatoric uncertainty?

No. It is noise in the data itself.

P7. A Gaussian process is queried far from all training points, with an RBF kernel of unit scale. What is its predictive variance?

About $k_{**} = 1$, because $k_* \approx 0$.

P8. A proposal has $\pi(y)/\pi(x) = 0.25$, and is symmetric. What is the acceptance probability?

0.25.